

Web Application Firewall

Project Documentation

Steps 1 & 2: Concepts & Existing Solutions

Cybersecurity Project — Academic Year 2024/2025

April 2025

1. Main Concepts of Web Application Firewalls

1.1 Definition & Purpose

A Web Application Firewall (WAF) is a security solution that monitors, filters, and blocks HTTP/HTTPS traffic between a web application and the Internet. Unlike traditional network firewalls that operate at the network and transport layers (L3/L4), a WAF operates at Layer 7 (the Application Layer) of the OSI model, giving it the ability to inspect the actual content of web requests and responses.

The primary purpose of a WAF is to protect web applications from common attack vectors by analyzing each request against a set of security rules. When a request violates a rule, the WAF can log, block, or sanitize the traffic before it reaches the application server.

1.2 Main Components

A WAF is composed of several interconnected components that work together to provide comprehensive protection:

1.2.1 Traffic Interception Engine

The traffic interception engine is the entry point of the WAF. It captures all incoming HTTP/HTTPS requests in real time before they reach the application server. This component is responsible for:

- Deep packet inspection (DPI) of request headers, body, cookies, and query strings
- SSL/TLS termination to decrypt HTTPS traffic for analysis
- Session tracking and management across multiple requests
- Protocol validation (ensuring requests conform to HTTP standards)

1.2.2 Rule Engine

The rule engine is the core decision-making component of the WAF. It evaluates each request against a set of predefined or custom security rules. Rule engines typically support:

- Signature-based detection: matching known attack patterns (e.g., OWASP CRS rules)
- Positive security model: only allowing known-good traffic patterns (allowlist)
- Negative security model: blocking known bad patterns (blocklist/denylist)
- Hybrid model: combining both approaches for stronger coverage

1.2.3 Anomaly Detection Module

Modern WAFs integrate anomaly scoring systems that evaluate the degree to which a request deviates from normal behavior. Rather than relying solely on exact signature matches, this module:

- Assigns anomaly scores to suspicious request attributes
- Accumulates scores across multiple rule matches within a single request

- Triggers a block action when the total score exceeds a configurable threshold
- Reduces false positives compared to binary match-or-block approaches

1.2.4 Logging & Alerting System

All WAF events — whether blocked, logged, or passed — are recorded for audit and forensic purposes. This component provides:

- Structured logging of blocked requests with timestamps, source IPs, and matched rules
- Real-time alerting via email, SIEM integration, or webhooks
- Dashboard visualization of traffic patterns and attack trends
- Support for compliance reporting (PCI-DSS, GDPR, ISO 27001)

1.2.5 Management Interface

The management interface allows administrators to configure rules, review logs, and tune WAF behavior. It typically includes:

- A web-based or CLI-based dashboard
- Rule management: create, update, enable/disable rules
- IP reputation and geolocation-based access control
- Reporting and analytics tools

1.3 Deployment Models

WAFs can be deployed in several architectural configurations, each with its own trade-offs:

Deployment Mode	Description	Use Case
Network-based	Hardware appliance placed inline in the network path. Offers low latency and high throughput.	Large enterprises with on-premises infrastructure
Host-based	Software installed on the same server as the application (e.g., ModSecurity as Apache module).	Single-server setups, cost-sensitive environments
Cloud-based	Managed WAF service delivered via a CDN or cloud provider (e.g., Cloudflare, AWS WAF).	SaaS applications, cloud-native architectures
Reverse Proxy	WAF sits as a reverse proxy in front of origin servers, intercepting all traffic.	Most common software-based WAF deployment

1.4 Functional Flow

The following describes the step-by-step processing pipeline of a WAF when handling an incoming HTTP request:

1. Client sends an HTTP/HTTPS request toward the web server.
2. The WAF intercepts the request before it reaches the application (acting as reverse proxy).
3. If HTTPS, SSL/TLS is terminated at the WAF to allow plaintext inspection.
4. The request is parsed: headers, URI, query parameters, cookies, and body are extracted.
5. The rule engine evaluates the request against all active security rules.
6. An anomaly score is computed based on rule matches.
7. Decision: if score exceeds threshold → block and log; otherwise → forward to application.
8. The WAF also inspects server responses (optional) to prevent data leakage.
9. Logs and metrics are stored and surfaced through the admin dashboard.

1.5 Common Attack Types Mitigated by WAFs

The OWASP Top 10 defines the most critical web application security risks. WAFs are specifically designed to detect and block the following:

Attack Type	Description	WAF Defense Mechanism
SQL Injection (SQLi)	Attacker injects malicious SQL code into input fields to manipulate the database.	Regex + keyword matching on query params and body
Cross-Site Scripting (XSS)	Malicious scripts injected into pages viewed by other users.	HTML entity detection, script tag filtering
Cross-Site Request Forgery (CSRF)	Forces users to perform unwanted actions on authenticated sites.	CSRF token validation, SameSite cookie checks
Path Traversal	Attacker navigates outside the intended directory using ../ sequences.	URI normalization and path sanitization rules
Remote/Local File Inclusion	Including unauthorized files in web application execution.	File extension and path-based detection rules
Command Injection	Injecting OS commands through vulnerable input fields.	Shell metacharacter detection (;, , &&, etc.)
HTTP Protocol Attacks	Malformed HTTP requests, HTTP smuggling, or header injection.	HTTP protocol compliance validation

1.6 Detection Techniques

Signature-Based Detection

This technique uses a database of known attack patterns (signatures) expressed as regular expressions or exact strings. When a request matches a signature, it is flagged. This is fast and reliable for known attacks, but ineffective against novel or zero-day exploits.

Behavioral / Anomaly-Based Detection

This technique establishes a baseline of normal traffic patterns and flags significant deviations. It is more effective against unknown attacks but requires a learning period and is prone to false positives during initial configuration.

Reputation-Based Detection

Requests originating from known malicious IP addresses, Tor exit nodes, or bot networks are blocked using IP reputation feeds. This provides an additional layer of protection before the rule engine is even invoked.

2. Overview of Existing WAF Solutions

2.1 Open-Source Solutions

2.1.1 ModSecurity

ModSecurity is the most widely deployed open-source WAF engine in the world. Originally developed as an Apache module, it has since evolved into a standalone library (libmodsecurity) with connectors for Apache, Nginx, and IIS.

Key characteristics:

- Rule language: Uses the SecLang rule language for defining detection logic
- OWASP CRS: Natively integrates with the OWASP Core Rule Set, the industry standard for WAF rules
- Deployment: Embedded as a module in the web server process
- Modes: Supports Detection Only, Prevention, and Learning modes
- Logging: Detailed audit logs with full request/response capture

Limitations: Complex configuration, high false-positive rate out of the box, and requires significant tuning for production environments.

2.1.2 OWASP Coraza WAF

Coraza is a modern, Go-based WAF engine designed as a high-performance successor to ModSecurity. It is fully compatible with the OWASP Core Rule Set and integrates natively with cloud-native environments.

- Written in Go for high concurrency and low memory footprint
- Supports Caddy, Nginx (via OpenResty), and Kubernetes ingress controllers
- WebAssembly (WASM) support for edge deployments
- Full OWASP CRS v4 compatibility

2.1.3 Nginx + Lua WAF (lua-resty-waf)

lua-resty-waf is a WAF built on top of OpenResty (Nginx + LuaJIT). It allows custom WAF logic to be written in Lua and executed at the Nginx request processing level.

- Extremely low latency due to Lua's JIT compilation
- Full access to Nginx internals and request lifecycle
- Flexible rule writing with Lua scripting
- Suitable for developers wanting fine-grained control

2.1.4 Shadow Daemon

Shadow Daemon is a modular WAF designed with a client-server architecture. The detection logic runs in a dedicated server process, decoupled from the web server. It supports PHP, Perl, and Python applications.

- Whitelist-based approach: learns the application's normal parameter structure
- Blacklist layer for known attack signatures
- Integrity model: verifies that parameters have not been manipulated
- Web-based management interface included

2.2 Commercial & Cloud-Based Solutions

2.2.1 Cloudflare WAF

Cloudflare WAF is a cloud-native, managed WAF solution delivered through Cloudflare's global CDN network. It provides enterprise-grade protection with minimal configuration.

- Anycast network: traffic is inspected at the closest data center to the client
- Managed rulesets: Cloudflare OWASP Core Ruleset, Bot Management, and DDoS protection
- Machine learning models: automatically detect novel attack patterns
- Zero-day response: Cloudflare deploys emergency rules within hours of new CVEs
- Global threat intelligence: shared signals across millions of protected domains

2.2.2 AWS WAF

AWS WAF is Amazon's native WAF service, tightly integrated with other AWS services such as CloudFront, ALB (Application Load Balancer), API Gateway, and AppSync.

- Web ACLs: define ordered rule groups applied to incoming requests
- AWS Managed Rules: pre-built rule groups for common threats (SQLi, XSS, known bad IPs)
- Rate-based rules: automatically block IPs exceeding request thresholds
- Integration with AWS Shield for advanced DDoS protection
- Pay-per-use pricing model based on rules and requests inspected

2.2.3 Imperva WAF (formerly Incapsula)

Imperva WAF is a leading enterprise WAF platform offering both cloud-based and on-premises deployments. It is widely used in financial services, healthcare, and government sectors.

- Proprietary threat intelligence database updated in real time
- Behavioral analysis with machine learning to detect account takeovers
- API security: automatic discovery and protection of REST and GraphQL endpoints
- Compliance automation for PCI-DSS, HIPAA, and SOC 2
- Advanced bot management with behavioral biometrics

2.2.4 Akamai Kona Site Defender

Kona Site Defender is Akamai's enterprise WAF solution delivered through their Intelligent Edge Platform, which spans over 4,000 locations worldwide.

- Delivered via Akamai's CDN, providing built-in DDoS absorption capacity
- Adaptive rate controls and client reputation scoring
- Scratchpad environment for testing rules without impacting production
- Integrated with Akamai's Bot Manager for advanced bot detection

2.2.5 F5 BIG-IP ASM

F5 BIG-IP Application Security Manager is a hardware/software WAF appliance primarily targeting large enterprises with complex, multi-tier application architectures.

- Full proxy architecture: terminates and re-originates all connections
- Learning mode: automatically builds a positive security policy from traffic analysis
- XML and JSON payload inspection for API security
- Integration with F5's iRules for custom traffic manipulation logic

2.3 Comparative Analysis

Solution	Type	Layer	OWASP CRS	ML/AI	Open Source	Best For
ModSecurity	Host-based	L7	Yes	No	Yes	Self-hosted apps
Coraza	Host-based	L7	Yes	No	Yes	Cloud-native, Go apps
Cloudflare WAF	Cloud	L7	Yes	Yes	No	SME to Enterprise
AWS WAF	Cloud	L7	Partial	Partial	No	AWS-hosted apps

Imperva WAF	Cloud/On-prem	L7	Yes	Yes	No	Enterprise, regulated sectors
F5 BIG-IP ASM	Appliance	L4-L7	Yes	Partial	No	Large enterprises
Shadow Daemon	Host-based	L7	No	No	Yes	PHP/Python apps

2.4 Positioning of Our Solution

Given the academic context of this project and the goal of demonstrating a deep understanding of WAF mechanics, our proposed solution is positioned as a custom-built, software-based WAF implemented as a reverse proxy. This approach is inspired by the architecture of ModSecurity and Coraza, but implemented from scratch to demonstrate mastery of the underlying concepts.

Our solution will differ from existing solutions in the following ways:

- Full transparency: every rule, scoring mechanism, and decision is visible and explainable
- Custom rule engine: built without relying on the OWASP CRS black box, to demonstrate understanding
- Real-time dashboard: a purpose-built admin interface for monitoring and rule management
- Extensibility: modular architecture allowing new detection modules to be plugged in